

AI-Assisted Design of SDR Systems

Roadmap v2.0: 14-week graduate course

Stack: Python / NumPy / PyTorch / ONNX Runtime

Training: Google Colab **Deployment:** Raspberry Pi 4 + RTL-SDR

Core outcome: A validated live modulation classifier

Course Contract and Learning Outcomes

Students enter with linear algebra, probability and DSP/RF fundamentals, but beginner Python and no deep-learning framework experience. W1–W2 explicitly teach the programming prerequisites. Mathematical rigor is assessed through gradient checks, tensor-shape reasoning, controlled experiments and systems measurements rather than repeated manual calculations for every architecture.

By completion, students can manipulate complex IQ data; train and diagnose a compact classifier; prevent capture leakage; compare IQ and STFT representations; preserve inference behavior through export; and operate a bounded streaming receiver with measured accuracy, coverage and latency.

Framework and scope. PyTorch is the instructional framework. Other frameworks may be used in optional conversion experiments; their presence in a Colab environment is not prohibited. Core deployment is PyTorch → ONNX → ONNX Runtime CPU on 64-bit Raspberry Pi OS. Native complex tensors support analysis; deployable models use validated real-valued operations. There is no required TensorFlow Serving, Android, browser inference, C++ OOT development, NLP or segmentation. AST transfer, denoising, occupancy forecasting and complex networks are optional extensions, not additional prerequisites for the capstone.

One progressive capstone. Introduce the receiver in W1, fix its classes and labeling protocol by W4, run replay inference on the Pi in W6, and require live integration by W12. The RTL-SDR is receive-only: before the course, the instructor arranges a suitable labeled RF source or independently identifiable broadcasts within the receiver’s tuning and bandwidth capabilities. Receive-only operation must comply with applicable local laws and band-specific reception restrictions; the instructor must verify that each selected signal may lawfully be received and recorded before assigning live capture. A replay demonstration supports reproducibility but does not replace live acquisition. Closed-loop transmitter adaptation is outside the supplied hardware scope.

Compute budget. Every experiment has a maximum cumulative training time of 7200 seconds on one T4, including resumed training and repeated seeds assigned to that experiment. The instructor benchmarks each required configuration before release, targeting at most 90 minutes and reserving margin for checkpointing. Starting limits are 100,000 training windows, 250,000 parameters, batch size 256 and 20 epochs; these are adjustable design limits, not measured runtime guarantees. Use early stopping and a wall-clock cutoff. Declare each ablation’s budget and the total lab workload; do not evade the limit by relabeling resumed runs. Log actual hardware and elapsed training time. Colab free does not guarantee a GPU or a T4: provide CPU smoke tests, reference checkpoints and an institutional fallback for required training. A reference-checkpoint exercise is not evidence that a student completed training.

Datasets and Evaluation Policy

- Use RadioML 2016.10A as the default initial benchmark with IQ layout $[N, 2, 128]$. An instructor-converted, verified NPZ avoids student handling of legacy pickle files. Preserve source checksum, labels and SNR metadata.

- A fixed RadioML 2018.01A HDF5 subset is an alternative, not a mandatory second benchmark. Freeze classes, examples per class/SNR, window length and array-axis conversion. Longer examples require separate runtime validation.
- Use instructor-generated GNU Radio streams and independently labeled RTL-SDR recordings for streaming, impairment and domain-shift experiments. Record sample rate, center frequency, gain, source/session identity and label intervals. Match bandwidth and samples-per-symbol assumptions where relevant.
- RadioML is an educational synthetic benchmark with known limitations. Example-level stratified splits do not prove independent-capture generalization. ORACLE is an optional transmitter-fingerprinting dataset, not a substitute for modulation labels; its documented binary-dtype correction must be respected.

Freeze train/validation/test indices before experimentation. For captures, split by independent session/source groups before windowing; related or overlapping windows cannot cross splits. Stratify the RadioML benchmark by modulation and SNR where capture groups are unavailable, and state that limitation. Fit any population statistics on training data only. Apply stochastic augmentation only to training data. SNR is evaluation metadata, not an input feature for the core classifier. Report accuracy and macro-F1 versus SNR, class counts and confusion matrices. Select models and rejection thresholds using validation data. Freeze the model before the final locked evaluation; any subsequent test-informed revision needs a new holdout or an explicit disclosure.

Simplified Reproducibility Manifest

Submit runnable code, one environment specification, data provenance and fixed splits, one run configuration, a checkpoint and measured results. No regulatory traceability matrix or release-approval workflow is a student prerequisite. Retain dataset attribution and applicable use terms in the README.

The following is an illustrative baseline to be installation-tested by the instructor before teaching. Colab does not read this file automatically: supply one bootstrap cell using the same package list and check runtime compatibility. The Pi uses only the inference dependencies plus an instructor-provisioned `librtlsdr`; it does not require the training stack. Record actual resolved versions on each platform, including the Pi OS image and acquisition library. GNU Radio is instructor-only tooling for offline synthetic-stream generation in a separately provisioned environment; instructors distribute the generated IQ files and metadata, so GNU Radio is intentionally absent from `environment.yml` and is not a runtime dependency of student notebooks or the Pi receiver. Students must not attempt `pipinstallgnuradio` as part of the course setup.

```
# environment.yml -- instructor validates before course release
name: sdr-ai
channels: [conda-forge]
dependencies:
  - python=3.11
  - pip
  - pip:
    - numpy==1.26.4
    - scipy==1.13.1
    - matplotlib==3.9.2
    - scikit-learn==1.5.2
    - torch==2.5.1
    - onnx==1.17.0
    - onnxruntime==1.20.1
```

```

- pyrtlsdr==0.3.0
variables:
  PYTHONHASHSEED: "42"
  CUBLAS_WORKSPACE_CONFIG: ":4096:8"

```

Three-step deterministic checklist.

1. Set environment variables before Python/CUDA initialization; seed Python, NumPy and PyTorch, enable deterministic algorithms and disable cuDNN benchmarking. Restart Colab when necessary. Unsupported deterministic operations must raise a visible error or be replaced; do not silently claim determinism.
2. Freeze splits and initialization; seed the DataLoader generator. Begin with `num_workers=0`. If workers are introduced, seed their random sources and validate ordering/sharding. Multiple workers are not inherently nondeterministic.
3. Repeat a short verification run on the same software/hardware and compare losses and weights. Save model, optimizer, epoch, RNG and loader-generator states for epoch-boundary recovery. Record performance-mode changes separately. Exact replay concerns fixed recorded IQ, not two different live RF acquisitions.

```

import random
import numpy as np
import torch
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.benchmark = False
generator = torch.Generator().manual_seed(42)

```

m0_1_environment_and_gpu_benchmark

The purpose is to establish, document, and experimentally verify a rock-solid, reproducible TensorFlow/Keras environment before any actual model training begins. It achieves this by:

- Pinning every dependency (Python packages, TensorFlow Serving image, TensorFlow.js, and Android LiteRT) to exact versions.
- Proving bitwise reproducibility across independent runs (via deterministic subprocesses)—essential for debugging.
- Proving GPU functionality through a rigorous performance benchmark, not just device detection (which can lie).
- Separating "deterministic verification" (for debugging) from "performance mode" (for training speed), since these require different TensorFlow configurations.
- Generating a complete audit trail of artifacts so the instructor/student can trace exactly what ran and where.

1. SET environment variables (KERAS_BACKEND, TF_CPP_MIN_LOG_LEVEL)
2. IMPORT libraries (os, json, subprocess, keras, tensorflow, numpy, matplotlib)

3. DEFINE constants (SEED, MATMUL_SIZE, REPETITIONS, WARMUPS)
4. CREATE artifact directory (./m0_1_artifacts)
5. [ENVIRONMENT CAPTURE]
 - RUN nvidia-smi to query GPU name, driver, memory
 - QUERY tensorflow.config for physical/logical GPUs
 - GET build info (CUDA, cuDNN versions)
 - WRITE environment_report.json
 - RUN pip freeze --all -> WRITE requirements.lock
 - WRITE tensorflow_serving_image.txt, package.json, libs.versions.toml (deployment pins)
 - WRITE dataset_provenance_template.json, pretrained_model_provenance_template.json
 - WRITE reproducibility_manifest.json (maps all artifacts)
6. [DETERMINISM VERIFICATION]
 - GENERATE a standalone probe script (determinism_probe.py) that:
 - * Sets KERAS_BACKEND, TF_DETERMINISTIC_OPS=1
 - * Initializes random weights and inputs with fixed seeds
 - * Runs a 2-layer forward pass, computes loss & gradients
 - * Serializes all numpy arrays to bytes and computes a SHA-256 fingerprint
 - LAUNCH the probe script in two isolated subprocesses (fresh kernels)
 - COMPARE fingerprints from run 1 and run 2
 - WRITE determinism_report.json (PASS if fingerprints match)
7. [CPU vs GPU PERFORMANCE BENCHMARK]
 - DEFINE timed_samples(operation) which warms up, synchronizes via .numpy(), records latency
 - ON CPU: create large (NxN) random matrices, time tf.matmul, record median latency
 - ON GPU: repeat the same timed operation on the GPU device
 - CALCULATE speedup = median_cpu / median_gpu
 - WRITE cpu_gpu_benchmark.json
8. [EXTENDED DIAGNOSTICS - Optional evidence]
 - BENCHMARK FP32 vs FP16 throughput and calculate numerical error
 - BENCHMARK CPU-resident operand vs GPU-resident operand (transfer cost)
 - BENCHMARK Conv2D throughput across batch sizes (1, 4, 8)
 - PLOT scaling graphs
9. [ACCEPTANCE REPORT]
 - CHECK all criteria (GPU visible, speedup > threshold, determinism matched, all locks created)
 - WRITE m0_1_acceptance_report.json and artifact_manifest.json
 - PRINT PASS/WAIT status for each check

Forced Failure & Bottleneck Diagnosis.

Goal: understand the statistical distribution of latencies (median, p95), and connect mathematical operation size to real-world hardware trade-offs—directly applicable to optimizing SDR real-time inference pipelines.

Task: Modify the notebook to deliberately break the GPU speedup condition, then diagnose why it broke using the recorded metrics. Instructions:

- Run the notebook as-is (with default MATMUL_SIZE=2048) and record the baseline GPU speedup.
- Modify the environment variable M0_1_MATMUL_SIZE from 2048 down to 32 (or 64) in the configuration cell.
- Rerun the entire notebook from a fresh kernel (this is critical—do not just re-run cells).

- Observe the final acceptance report—it will now fail the "gpu_speedup_demonstrated" check (because the matrix is too small for the GPU to overcome kernel launch overhead).
- Write a short post-mortem (3-4 sentences) answering: What were the CPU and GPU median latencies at size 32?

Why does the speedup drop below 1.0x, even though TensorFlow detects the GPU?

If you were building an SDR model with IQ samples of shape (batch=32, time_steps=1024), would you prefer CPU or GPU for this size? Justify using the benchmark data.

How does the float16 error magnitude change when you reduce the matrix size? Why might this matter for phase-sensitive RF signals?

- Re-run the determinism probe with TF_DETERMINISTIC_OPS disabled (remove it from the child environment). Show that the two subprocesses still produce the exact same fingerprint due to seeding (if using stateless ops), but note that in real training with tf.function autotuning, non-determinism may creep in.

Failure

Actionable Plan: W1–W14

Week	Focus / milestones	Required result
W1	Python/NumPy through DSP; capstone introduction (Pre-M0)	Load IQ, manipulate arrays, compute power and explain shapes.
W2	Tensors, autograd and environment checks (M0)	Toy IQ model, gradient check and short Pi capture.
W3	Minimal harness and supplied CNN (M1.1–M1.2)	Single-batch overfit, configuration, logs and checkpoint recovery.
W4	RadioML baseline and ablations (M1.2–M1.3)	Fixed splits, per-SNR baseline results and capstone class selection.
W5	IQ preprocessing and augmentation (M2.1–M2.2)	Correct data pipeline and bounded, label-preserving impairments.
W6	ONNX export and early Pi inference (M1.4/M6.1)	Export parity and recorded-IQ inference on Pi.
W7	Acquisition, buffering and stateful DSP (M2.3/M6.1)	Continuous capture/replay with queue and drop measurements.
W8	STFT and spectrogram CNN (M3.1–M3.2)	Fair IQ-versus-spectrogram comparison including computational cost.
W9	Capture evaluation and adaptation (M4; M3.3 optional)	Independent-session validation; complex layers only if time permits.
W10	RF diagnostic drills and selection (M5.1)	Isolated failure causes and validated deployment candidate.
W11	Training and Pi profiling (M5.2/M6.2 optional)	Measured bottlenecks; optional mixed precision/INT8 with accuracy checks.
W12	Complete live integration (M6.1/M6.3)	Working receiver at declared cadence and sustained-operation test.
W13	Robustness, corrections and freeze (M6.4/M7)	Locked evaluation, latency distributions and documented operating limits.
W14	Final capstone assessment (M7)	Independently labeled live demonstration, replay and engineering defense.

Ordering and Instructor Preparation

M0 precedes training; M1.3 precedes representation comparisons; M1.4 opens early deployment. M2.3 and M4 are prerequisites for final live claims. M5 and M6 converge on M7. Optional extensions never block the core capstone. Supply the initial CNN, environment bootstrap, bounded dataset, receiver skeleton and labeled signal source; students implement and defend the specified learning objectives rather than unrelated infrastructure.

Pre-M0 — Python through IQ (W1)

Goal. Close the programming gap using familiar signals.

Pre-M0.1. Functions, indexing, slicing, broadcasting, dtypes, file loading, plotting and joint IQ power. Defer custom classes until needed.

Artifact. Notebook loading IQ, plotting time traces and constellations, computing power and documenting dimensions; one-paragraph capstone description.

Pass criterion. Students independently explain array axes and correct an injected axis swap. Power agrees with a direct complex-magnitude calculation.

M0 — Tensors and Gradients (W2)

Goal. Connect mathematical gradients to executable training.

M0.1. Validate the Colab environment with a short computation; check Pi connectivity and acquire a short IQ recording. Log hardware and versions.

Artifact. Environment report and readable capture with acquisition metadata.

Pass criterion. The computation produces finite outputs and the recording is readable. A GPU check is not a guarantee of future Colab availability.

M0.2. Train a small IQ-feature MLP using tensor operations, autograd and explicit SGD updates. Inspect gradient shapes and perform finite differences.

Artifact. Training loop, chain-rule explanation and one manual parameter count.

Pass criterion. Loss decreases; selected gradients match finite differences within declared tolerances; parameter shapes and count are correct.

M0.3. Reproduce the toy model using a minimal `nn.Module` and optimizer. Introduce only the class syntax needed for this model.

Pass criterion. Identical weights and batches produce matching forward outputs and one controlled SGD update.

M1 — Training and Baselines (W3–W4; export W6)

Goal. Create one observable experiment path.

M1.1 — W3. A minimal harness reads one configuration, logs training and validation metrics, and saves epoch-boundary checkpoints. A supplied compact CNN separates training-loop learning from architecture design.

Artifact. Configuration, learning curves, checkpoint and short recovery test.

Pass criterion. Resume restores model, optimizer, completed epoch and random state. Validation uses evaluation mode without parameter updates. Arbitrary mid-batch interruption equivalence is not required.

M1.2 — W3/W4. Overfit one batch, then change one factor at a time: learning rate, regularization or training SNR distribution. Keep evaluation fixed.

Artifact. Sanity-check result and a bounded ablation table with runtime.

Pass criterion. Single-batch classification succeeds with low loss and finite gradients; interpretations distinguish SNR-stratified evaluation from changing the training distribution. Failed experiments receive a controlled diagnosis.

M1.3 — W4. Train a two-channel RadioML Conv1D baseline and compare against a DSP-feature classifier using identical splits. Predeclare architecture, metric, budget and validation target. Choose capstone classes and live labels.

Artifact. Class map, split indices, tensor/parameter summary, per-SNR metrics, confusion matrices and a capstone input/label contract.

Pass criterion. Run stays within budget and is compared fairly with the baseline. Do not require arbitrary universal accuracy across low-SNR examples. Test data remain locked until final selection.

M1.4 — W6. Export the baseline in evaluation mode to ONNX and run fixed recorded examples on both the training host and Pi CPU.

Artifact. ONNX model, golden IQ batch, class/preprocessing contract and parity report.

Pass criterion. Logits match within declared absolute/relative tolerances, the Pi client needs no training-model definition, and batch-one latency is measured.

M2 — IQ Pipelines and Streaming (W5/W7)

Goal. Preserve data semantics offline and live.

M2.1 — W5. Validate IQ order, shape, labels, sample-rate assumptions and normalization. Use a map-style DataLoader for finite data. Compare a baseline and optimized loader only with equivalent data and preprocessing semantics.

Artifact. Pipeline checks and a small throughput/memory benchmark.

Pass criterion. No split leakage or label changes; measured improvement is not mandatory. Cache deterministic work, not a fixed realization of random augmentation.

Note. Per-window joint-power normalization uses the current window only; it does not fit dataset statistics. Preserve received power separately for signal detection. Never normalize I and Q independently without justification.

```
# Instructor NPZ preserves original examples and class/SNR mapping.
from torch.utils.data import TensorDataset, DataLoader
d = np.load("radioml2016_course.npz", allow_pickle=False)
X = d["iq"].astype(np.float32)          # [N,2,128]
y = d["label"].astype(np.int64)
idx = np.load("split_indices.npz")["train"]
assert X.shape[1:] == (2, 128)

def normalize_iq(x):
    p = np.mean(np.sum(x*x, axis=-2, keepdims=True),
                axis=-1, keepdims=True)
    return x / np.sqrt(np.maximum(p, 1e-8))

ds = TensorDataset(torch.from_numpy(normalize_iq(X[idx])),
                   torch.from_numpy(y[idx]))
loader = DataLoader(ds, batch_size=256, shuffle=True,
                   num_workers=0, generator=generator)
```

M2.2 — W5. Apply training-only phase rotation, bounded CFO, noise and channel impairments. Preserve modulation identity and explain each bound.

Pass criterion. Students verify the power/noise model and label semantics. Adding noise to already noisy IQ does not establish a new exact SNR without knowledge of the underlying signal and noise powers.

M2.3 — W7. Connect continuous acquisition or replay to stateful channel selection, filtering/resampling, window assembly and a bounded queue. Record session metadata before windowing; retain filter state across chunks.

Artifact. Chunk-safe pipeline, ring-buffer implementation, queue/drop logs and replay comparisons using different chunk sizes.

Pass criterion. Equivalent replay streams produce equivalent windows despite chunk boundaries. No accidental duplication occurs; discontinuities reset state explicitly. Acquisition and inference run independently. Deliberate inference subsampling is distinguished from lost acquisition samples.

Note. An IterableDataset is optional for actual streams, not required for finite RadioML arrays. Training prefetch and receiver buffering solve different problems and have different correctness criteria.

M3 — RF Representations (W8; optional extension W9)

Goal. Compare representations within deployment limits.

M3.1. Refine the compact real two-channel Conv1D baseline with small kernels and global pooling. Explain receptive field and parameter cost.

M3.2. Compare with a small Conv2D on two-sided complex-STFT log power. Fix FFT length, window, hop, centering, scaling and frequency ordering.

Artifact. Matched-data comparison of per-SNR metrics, parameters, training time, preprocessing cost and Pi inference time.

Pass criterion. Use identical source examples/splits and declared budgets. Discuss discarded phase information; a spectrogram need not outperform IQ. Do not concatenate unrelated benchmark windows into fictitious continuous signals.

```
# iq is a normalized real tensor [B,2,128].
z = torch.complex(iq[:, 0], iq[:, 1])
S = torch.stft(z, n_fft=32, hop_length=8,
               window=torch.hann_window(32, device=z.device),
               center=False, onesided=False, return_complex=True)
features = torch.log10(torch.fft.fftshift(
    S.abs().square(), dim=-2).clamp_min(1e-8)).unsqueeze(1)
# Deployment must reproduce this preprocessing; verify golden features.
```

M3.3 — Optional P2. Implement tied real convolutions representing complex multiplication: $Y_r = W_r * I - W_i * Q$, $Y_i = W_r * Q + W_i * I$, where $*$ denotes convolution. Compare with unconstrained real two-channel convolution.

Pass criterion. Verify shapes and gradients, explain weight coupling, and check export support. This extension cannot displace required capture evaluation.

M4 — Capture-Domain Evaluation (W9)

Goal. Measure and address synthetic-to-real mismatch.

M4.1. Evaluate candidate models on independently labeled captures. Keep source/session groups separate; inspect gain, bandwidth, sample-rate, frequency-offset and channel differences. Do not infer ground truth from model output.

Artifact. Synthetic-versus-capture validation report and documented acquisition conditions.

M4.2. Perform limited adaptation on training captures, using validation captures for selection. Add no-signal handling and a validation-tuned rejection rule.

Pass criterion. Report held-out session results at final evaluation; report rejection coverage, false rejections and false acceptance on selected unfamiliar signals. Low confidence is a heuristic, not proof of general open-set detection.

M5 — Diagnostics and Profiling (W10/W11)

Goal. Diagnose before optimizing.

M5.1 — W10. Diagnose injected RF failures: SNR leakage, swapped IQ, clipping, inconsistent scaling, invalid power normalization or gradient failures. Naive subtraction across the phase wrap boundary is a useful defect; wrapped phase itself is not automatically wrong.

Artifact. Symptom, hypothesis, isolating experiment, correction and post-fix result.

Pass criterion. Evidence identifies the cause rather than merely correlating a histogram with it. Select the deployment candidate using validation metrics and Pi resource measurements, not the locked test.

M5.2 — W11. Profile training input wait, computation and memory; separately profile receiver DSP, queuing and inference. Test mixed precision only if justified by a measured T4 bottleneck.

Artifact. Stage timings and optional FP32/mixed-precision comparison with accuracy.

Pass criterion. Include warm-up and appropriate GPU synchronization. Explain neutral or negative optimization results. GPU utilization alone does not establish whether a workload is compute- or memory-bound.

M6 — Pi Deployment and Integration (W6–W13)

Goal. Deliver a bounded, observable receiver.

M6.1 — W6/W7/W12. Use the early exported baseline and replay client to integrate live `pyrtlsdr` acquisition, stateful DSP and ONNX Runtime CPU. By W12 produce classes, confidence/rejection and timestamps continuously.

Artifact. Model, preprocessing/class contract, runnable receiver and logs.

Pass criterion. A ten-minute sustained test has bounded memory and backlog. Record actual acquisition rate, processed rate, decision cadence, drops and coverage. The GUI is outside the critical path and receives decimated display data.

```
# Consumer after stateful DSP/windowing; acquisition runs independently.
import onnxruntime as ort
session = ort.InferenceSession("model.onnx",
    providers=["CPUExecutionProvider"])
def classify(window):
    # real [2,L], selected IQ model
    x = normalize_iq(window.astype(np.float32))[None]
    return session.run(None, {"iq": x})[0]
# Export must name the input "iq"; load the saved class map separately.
```

M6.2 — Optional W11. Compare supported INT8 inference against the working FP32 model on the actual Pi. Use representative training data for calibration; evaluate per-class/per-SNR accuracy and latency.

Pass criterion. Record model size, end-to-end timing and accuracy changes. Neither quantization speedup nor FP16 CPU benefit is presumed. ONNX-to-TF Lite and Edge TPU are separate optional branches with additional compatibility and hardware requirements, not the core pipeline.

M6.3 — W12. Complete integration, including exception handling, input validation, queue limits and a documented restart procedure.

Pass criterion. Replay and live processing share the same input contract. Disconnections and missing samples produce explicit status rather than stale predictions.

M6.4 — W13. Measure robustness and end-to-end p50/p95/p99 latency, including acquisition/window accumulation, DSP, queuing and inference. Record timestamp definitions, sample count, RAM and thermal/throttling conditions.

Pass criterion. For processed sample rate f_s and hop H , compare service capacity against f_s/H windows/s. Declare any intentional coverage reduction. Use repeatable in-band impairment tests; distinguish

receiver overload from in-band interference. Commercial Wi-Fi/LTE is not an assumed compatible test source.

M7 — Progressive Capstone (W1–W14)

Goal. Defend a reproducible live modulation classifier.

M7.1. W1: introduce objective; W4: freeze classes and label protocol; W6: Pi replay; W12: integrated live receiver; W13: correct using development data, freeze model and run locked evaluation; W14: demonstrate and defend.

Artifact. Repository with README, environment/run records, data provenance and splits, training code/checkpoint, ONNX model, receiver, replay sample, metrics and a concise engineering report. Avoid duplicating artifacts already produced.

Evaluation criteria. Assess independent labels and split integrity; model-versus-baseline results; per-class/per-SNR performance where SNR is known; capture-domain gaps; rejection behavior; export parity; throughput, latency and coverage; failure analysis; and reproduction from the submitted instructions. Do not invent exact SNR labels for uncalibrated live captures.

Pass criterion. The frozen system classifies the declared live signal set and reproduces a fixed replay evaluation. Students explain representation, parameter cost, training budget and deployment choices. Report limitations honestly; the final week is an assessment of an already integrated system.

Technical References

- DeepSig: dataset descriptions and limitations.
- ORACLE: fingerprinting task and dtype correction.
- PyTorch reproducibility.
- ONNX Runtime Python and CPU installation.
- pyrtlsdr acquisition interface.
- Colab resource availability.